

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of THURSDAY 5 JUNE 2025 – Laboratory Part

WITH SOLUTIONS

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the ability to test if **all elements of an array satisfy a certain logical condition**.

This is done by a **new expression operator** named **`all`** whose syntax appears like this:

$$\text{all}(\langle array \rangle, \langle var \rangle, \langle exp \rangle)$$

where $\langle array \rangle$ is the **identifier of an array**, $\langle var \rangle$ is the **identifier of a variable**, and $\langle exp \rangle$ is a **generic expression**.

In particular, $\langle array \rangle$ is the array whose elements need to be tested, and $\langle var \rangle$ is a variable — previously declared in the program — which may appear in the expression $\langle exp \rangle$. Each element in $\langle array \rangle$ is checked by assigning it to $\langle var \rangle$ (overwriting its previous value) and then evaluating the expression ($\langle exp \rangle$). If the expression is equal to zero for at least one array element, the result of the operator is zero. Otherwise, no array elements make the expression equal to zero, and the result of the operator is one. Once $\langle exp \rangle$ is found to evaluate to zero for some array element, the remaining array elements that have not been checked can be skipped. The order in which the array elements are checked is also undefined.

If $\langle array \rangle$ is not an array, or if $\langle var \rangle$ is not a scalar, compilation shall fail with an error. `all` can be nested. There is no special handling required if the expression $\langle exp \rangle$ does not contain any occurrence of $\langle var \rangle$. The variable $\langle var \rangle$ may occur multiple times in the expression, and at the end of the operations performed by the operator its value is **undefined**.

The following example shows the behavior of the statement.

```
int a[10], b[10], v, e, i;

// Fill 'a' with the numbers from 1 to 10 and 'b' with 20s
i = 0;
while (i < 10) {
    a[i] = i+1;
    b[i] = 20;
    i = i + 1;
}

// Prints 1 because all elements of 'a' are greater than 0
write(all(a, e, e>0));
// Prints 0 because a[4] to a[9] are not less than 5
write(all(a, e, e<5));
// Prints 3 because all() evaluates to 0 and the value of the
// expression is therefore 3
write(3+all(a, e, e>5)/2);
// Prints 0 because we are using the values from a to read from
// b, and all values in b are greater than 15
write(all(a, e, b[e-1]<15));
```

- (a) Define the tokens (and the related declarations in **`scanner.l`** and **`parser.y`**). (1 point)
 - (b) Define the syntactic rules or the modifications required to the existing ones. (2 points)
 - (c) Define the semantic actions needed to implement the required functionality. (22 points)
-

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/parser.h b/acse/parser.h
index 100112f..f9706bd 100644
--- a/acse/parser.h
+++ b/acse/parser.h
@@ -26,6 +26,13 @@ typedef struct {
    t_label *lExit; ///< Label to the first instruction after the loop.
} t_whileStmt;

+typedef struct {
+  t_label *lLoop;
+  t_label *lExit;
+  t_regID rIdx;
+  t_regID rRes;
+} t_allExpr;
+
+
/**
 * @}
 */
diff --git a/acse/parser.y b/acse/parser.y
index 4039075..3bab379 100644
--- a/acse/parser.y
+++ b/acse/parser.y
@@ -50,6 +50,7 @@ void yyerror(const char *msg)
    t_label *label;
    t_ifStmt ifStmt;
    t_whileStmt whileStmt;
+  t_allExpr allExpr;
}

/*
@@ -77,6 +78,7 @@ void yyerror(const char *msg)
%token <label> DO
%token <string> IDENTIFIER
%token <integer> NUMBER
+%token <allExpr> ALL

/*
 * Non-terminal symbol semantic value type declarations
@@ -435,6 +437,59 @@ exp
    $$ = getNewRegister(program);
    genOR(program, $$, rNormalizedOp1, rNormalizedOp2);
}
+ | ALL LPAR var_id COMMA var_id COMMA
+ {
+   // The first argument must be an array, the second must be a scalar.
+   if (!isArray($3) || isArray($5)) {
+     yyerror("type mismatch");
+     YYERROR;
+   }
+
+   // Reserve a register for the result and generate an LI instruction to
+   // initialize it to 1 (true).
+   $1.rRes = getNewRegister(program);
+   genLI(program, $1.rRes, 1);
+   // Reserve a register for the array index we are accessing and loop counter
+   // and generate an LI to initialize it to zero.
+   $1.rIdx = getNewRegister(program);
```

```

+   genLI(program, $1.rIdx, 0);
+   // Generate a label for the back-edge of the loop.
+   $1.lLoop = createLabel(program);
+   assignLabel(program, $1.lLoop);
+   // Generate code to check if the array index reached the end of the array
+   // and break out of the loop (without setting the result to zero)
+   // in that case.
+   t_regID rMax = getNewRegister(program);
+   genLI(program, rMax, $3->arraySize);
+   $1.lExit = createLabel(program);
+   genBGE(program, $1.rIdx, rMax, $1.lExit);
+   // Generate code that loads the array element at the current index and
+   // stores it to the variable.
+   t_regID rElem = genLoadArrayElement(program, $3, $1.rIdx);
+   genStoreRegisterToVariable(program, $5, rElem);
+   // (the code generated up to here appears before the code that evaluates
+   // the expression)
+ }
+ exp RPAR
+ {
+   // (the code generated from here appears after the code that evaluates the
+   // expression)
+   // Generate the increment of the array index / loop counter.
+   genADDI(program, $1.rIdx, $1.rIdx, 1);
+   // Generate code that checks the expression's last value, and if it was
+   // not equal to zero (true) continue with the next loop iteration.
+   genBNE(program, $8, REG_0, $1.lLoop);
+   // If the expression's last value was false, and the result of the operator
+   // is zero (false), the branch generated above is not taken. Generate an
+   // instruction (that appears after the branch) that sets the result to
+   // zero. We are now out of the loop effectively.
+   genLI(program, $1.rRes, 0);
+   // Generate the label for exiting the loop without setting the result to
+   // zero.
+   assignLabel(program, $1.lExit);
+   // Set the result register of the expression.
+   $$ = $1.rRes;
+ }
+ ;

```

```

var_id
diff --git a/acse/scanner.l b/acse/scanner.l
index 3d35cd5..57507ee 100644
--- a/acse/scanner.l
+++ b/acse/scanner.l
@@ -81,6 +81,7 @@ ID [a-zA-Z_][a-zA-Z0-9_]*
"return" { return RETURN; }
"read" { return READ; }
"write" { return WRITE; }
+"all" { return ALL; }

{ID} {
    yylval.string = strdup(yytext);
diff --git a/tests/all/all.src b/tests/all/all.src
new file mode 100644
index 0000000..29b03f2
--- /dev/null
+++ b/tests/all/all.src
@@ -0,0 +1,20 @@
+int a[10], b[10], v, e, i;
+
+// Fill 'a' with the numbers from 1 to 10 and 'b' with 20s
+i = 0;
+while (i < 10) {

```

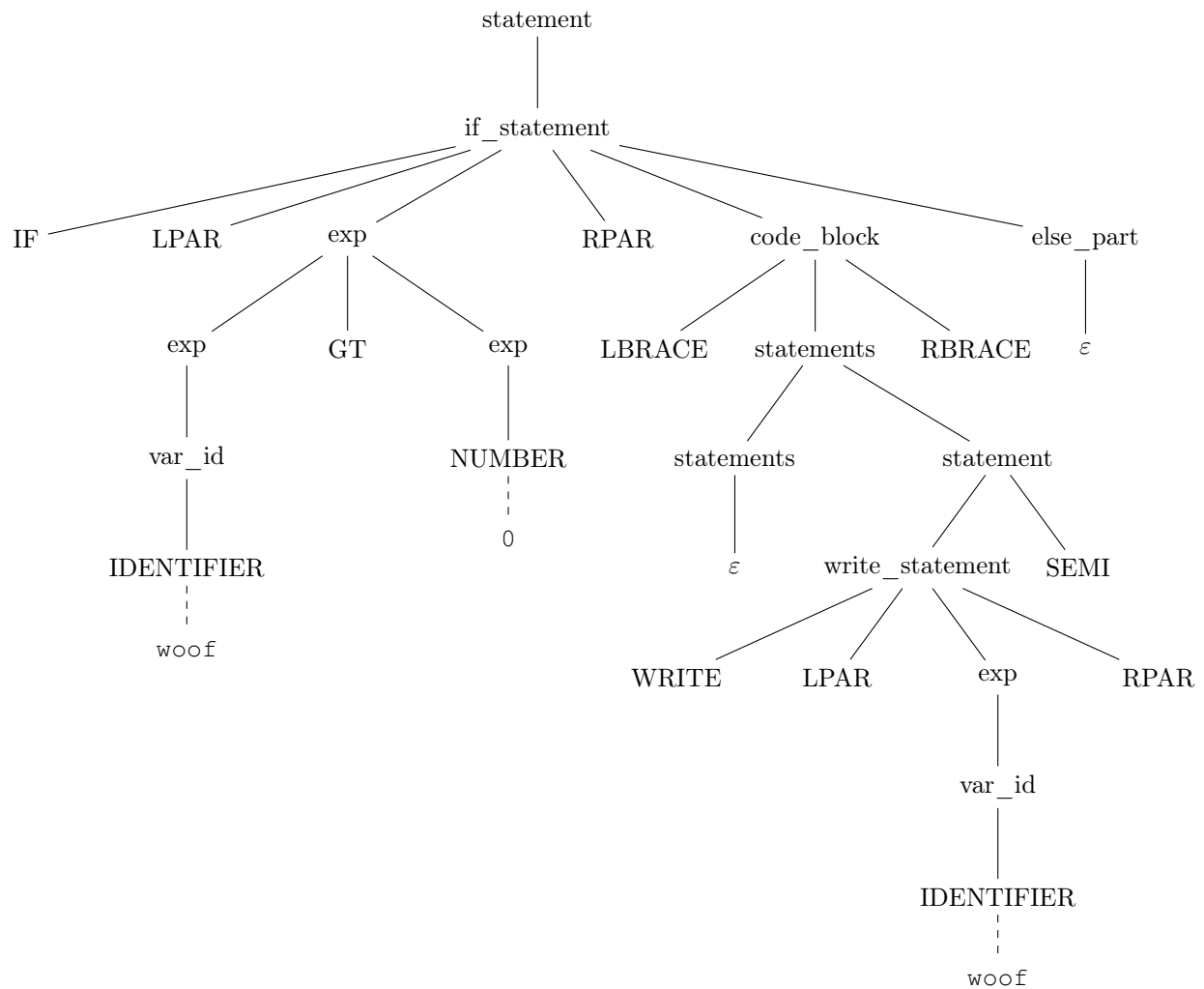
```
+  a[i] = i+1;
+  b[i] = 20;
+  i = i + 1;
+}
+
+// Prints 1 because all elements of 'a' are greater than 0
+write(all(a, e, e>0));
+// Prints 0 because a[4] to a[9] are not less than 5
+write(all(a, e, e<5));
+// Prints 3 because all() evaluates to 0 and the value of the
+// expression is therefore 3
+write(3+all(a, e, e>5)/2);
+// Prints 0 because we are using the values from a to read from
+// b, and all values in b are greater than 15
+write(all(a, e, b[e-1]<15));
```

2. Given the following LANCE code snippet:

```
if (woof>0) {write(woof); }
```

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` starting from the statement nonterminal. (5 points)

Solution



2 Bonus Question (5 points)

Show how to modify your previous implementation of the `all` operator into the **any** operator. `any` has a value of 1 if at least one array element makes the expression true. Write in detail which lines of code are changed and how.
